EarthCARE ESA MAAP Data Access Example

Contents

- EarthCARE ESA MAAP Data Access Example
- Stream and plot data
- Download data

@ ESA, 2025 = Licensed under “European Space Agency Community License”

Author: Saskia Brose (saskia.brose@esa.int)

Created: 05-11-2025

Updates:

- Long lasting token
- JSON keys for EarthCARE data files renamed
- [18/02/2025] Correction to collection description

This script shows how to query the ESA MAAP catalog, stream/download EarthCARE data and then plot the cloud water path and cloud top tempetaure directly from the streamed file.

Prerequisities

```
from pystac_client import Client
import fsspec
import xarray as xr
import matplotlib.pyplot as plt
from tqdm import tqdm
import requests
from IPython.display import Image, display
import pathlib
```

Using the STAC API to query the ESA MAAP stac catalog

While the discovery of data (querying the ESA MAAP catalogue) does not require any authentication or authorization, accessing the data requires a token generated with an authorized ESA account (EO Sign in) to verify the user. This is the same account and credentials you will have used for the OADS system.

```
catalog_url = 'https://catalog.maap.eo.esa.int/catalogue/'
catalog = Client.open(catalog_url)
```

The EarthCARE collections have the same name as previously on OADS, but have the extension `_MAAP` to distinguish them from the OADS collections. Currently the latest two baselines are provided and there are 12 collections:

Restricted to special users	Restricted CalVal users	Open and Free
EarthCAREL1InstChecked_MAAP	EarthCAREL0L1Products_MAAP	EarthCAREL1Vali
EarthCAREL2InstChecked_MAAP	EarthCAREL2Products_MAAP	EarthCAREL2Val
JAXAL2InstChecked_MAAP	EarthCAREAuxiliary_MAAP	JAXAL2Validatec
EarthCAREOrbitData_MAAP	JAXAL2Products_MAAP	
EarthCAREXMETL1DProducts10_MAAP		

The **first** step is to select a collection. You can select multiple collections by extending the list. Note that if you pass multiple collections, the query filters you use must exist across both collections!

```
# Select one or more collection(s)
EC_COLLECTION = ['EarthCAREL2Validated_MAAP']
#EC_multiple_COLLECTIONS = ['EarthCAREL2Validated_MAAP', 'EarthCAREL1Validated
```

The **second** step is to further narrow down your search:

Datetime represents the temporal coverage of the data. None can be used for both start and end to indicated unbounded queries.

bbox is defined by the bottom left corner (longmin latmin) and the top right corner coordinates (longmax latmax). The default is [-180,-90,180,90].

Filter – allows you to search based on different metadata parameters.
To understand which queryables exist, you can visit:
`https://catalog.maap.eo.esa.int/catalogue/collections/NAMEOFCOLLECTION/queryables`

Multiple filters can be combined with *or* and *and* following a boolean type of logic.

Examples include:

- productType
- frame
- processingLevel
- instrument
- orbitNumber

```
search = catalog.search(
    collections=EC_COLLECTION,
    filter="productType = 'MSI_COP_2A' and orbitNumber > 5000", # For example
    bbox = [0, -20, 10, -10], # Filter by area
    datetime = ['2025-10-01T00:00:00Z', '2025-12-31T23:59:59Z'], # Filter by d
    method = 'GET', # This is necessary
    max_items=5  # Adjust as needed, given the large amount of products it is
)
items = list(search.items()) # Get all items as a list
results = search.matched()

print(f"{results} items found that matched the query.")
print(f"Accessing {len(items)} items (limited by max_items).")
```

```
107 items found that matched the query.
Accessing 5 items (limited by max_items).
```

```
# OPTIONAL: Inspect results
for item in items:
    print(f"Item ID: {item.id}")
    print(f"  - Datetime: {item.datetime}")
    print(f"  - BBOX: {item.bbox}")
    print(f"  - Properties: {item.properties}")
    print(f"  - Assets: {list(item.assets.keys())}")
    print()
```

```
Item ID: ECA_EXBA_MSI_COP_2A_20251002T132907Z_20251002T193545Z_07649E
  - Datetime: 2025-10-02 13:29:07+00:00
  - BBOX: [0.556153, -22.683256, 10.825838, 22.560616]
  - Properties: {'start_datetime': '2025-10-02T13:29:07.000Z', 'end_datetime':
  - Assets: ['thumbnail', 'product', 'metadata_ogc_10_157r4', 'enclosure_h5',

Item ID: ECA_EXBA_MSI_COP_2A_20251009T133627Z_20251009T151530Z_07758E
  - Datetime: 2025-10-09 13:36:27+00:00
  - BBOX: [-1.289523, -22.715305, 8.997155, 22.588572]
  - Properties: {'start_datetime': '2025-10-09T13:36:27.000Z', 'end_datetime':
  - Assets: ['thumbnail', 'product', 'metadata_ogc_10_157r4', 'enclosure_h5',

Item ID: ECA_EXBA_MSI_COP_2A_20251004T131758Z_20251004T162733Z_07680E
  - Datetime: 2025-10-04 13:17:58+00:00
  - BBOX: [3.332371, -22.724861, 13.615556, 22.586065]
  - Properties: {'start_datetime': '2025-10-04T13:17:58.000Z', 'end_datetime':
  - Assets: ['thumbnail', 'product', 'metadata_ogc_10_157r4', 'enclosure_h5',

Item ID: ECA_EXBA_MSI_COP_2A_20251006T130640Z_20251006T205725Z_07711E
  - Datetime: 2025-10-06 13:06:40+00:00
  - BBOX: [6.166287, -22.691391, 16.435532, 22.5571]
  - Properties: {'start_datetime': '2025-10-06T13:06:40.000Z', 'end_datetime':
  - Assets: ['thumbnail', 'product', 'metadata_ogc_10_157r4', 'enclosure_h5',

Item ID: ECA_EXBA_MSI_COP_2A_20251008T125531Z_20251008T173823Z_07742E
  - Datetime: 2025-10-08 12:55:31+00:00
  - BBOX: [8.943265, -22.726088, 19.229439, 22.576187]
  - Properties: {'start_datetime': '2025-10-08T12:55:31.000Z', 'end_datetime':
  - Assets: ['thumbnail', 'product', 'metadata_ogc_10_157r4', 'enclosure_h5',
```

```
# Choose one item
ec_product = items[1]
```

Results

Assets in the ESA MAAP STAC Catalog: Each granule (one frame of EarthCARE data per product) includes multiple **assets**, which are different files that serve distinct purposes. These assets can include preview images, scientific data, metadata, and more.

```
# Optional: Inspect available assets
for asset_key, asset in ec_product.assets.items():
        print(f"  Asset key: {asset_key}")
        print(f"    href: {asset.href}")
        print(f"    type: {asset.media_type}")
        print(f"    roles: {asset.roles}")
```

```
Asset key: thumbnail
  href: https://catalog.maap.eo.esa.int/data/earthcare-pdgs-01/EarthCARE/MSI_
  type: image/jpeg
  roles: ['thumbnail']
Asset key: product
  href: https://catalog.maap.eo.esa.int/data/zipper/earthcare-pdgs-01/EarthC
  type: application/zip
  roles: ['data', 'metadata', 'archive']
Asset key: metadata_ogc_10_157r4
  href: https://catalog.maap.eo.esa.int/catalogue/collections/EarthCAREL2Val
  type: application/gml+xml;profile="http://www.opengis.net/spec/EOMPOM/1.1"
  roles: ['metadata']
Asset key: enclosure_h5
  href: https://catalog.maap.eo.esa.int/data/earthcare-pdgs-01/EarthCARE/MSI_
  type: application/x-hdf5
  roles: ['data']
Asset key: metadata_ogc_17_003r2
  href: https://catalog.maap.eo.esa.int/catalogue/collections/EarthCAREL2Val
  type: application/geo+json;profile="http://www.opengis.net/spec/eo-geojson
  roles: ['metadata']
Asset key: metadata_iso_19139
  href: https://catalog.maap.eo.esa.int/catalogue/collections/EarthCAREL2Val
  type: application/vnd.iso.19139+xml
  roles: ['metadata']
Asset key: quicklook_jpeg
  href: https://catalog.maap.eo.esa.int/data/earthcare-pdgs-01/EarthCARE/MSI_
  type: image/jpeg
  roles: ['overview']
Asset key: enclosure_hdr
  href: https://catalog.maap.eo.esa.int/data/earthcare-pdgs-01/EarthCARE/MSI_
  type: image/vnd.radiance
  roles: ['data']
```

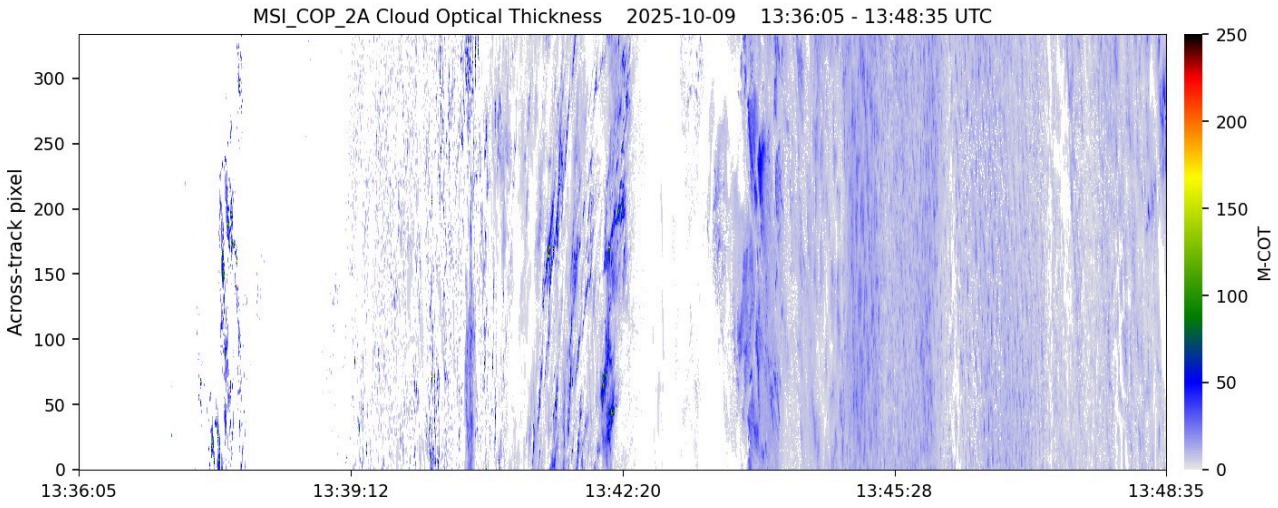
Tips:

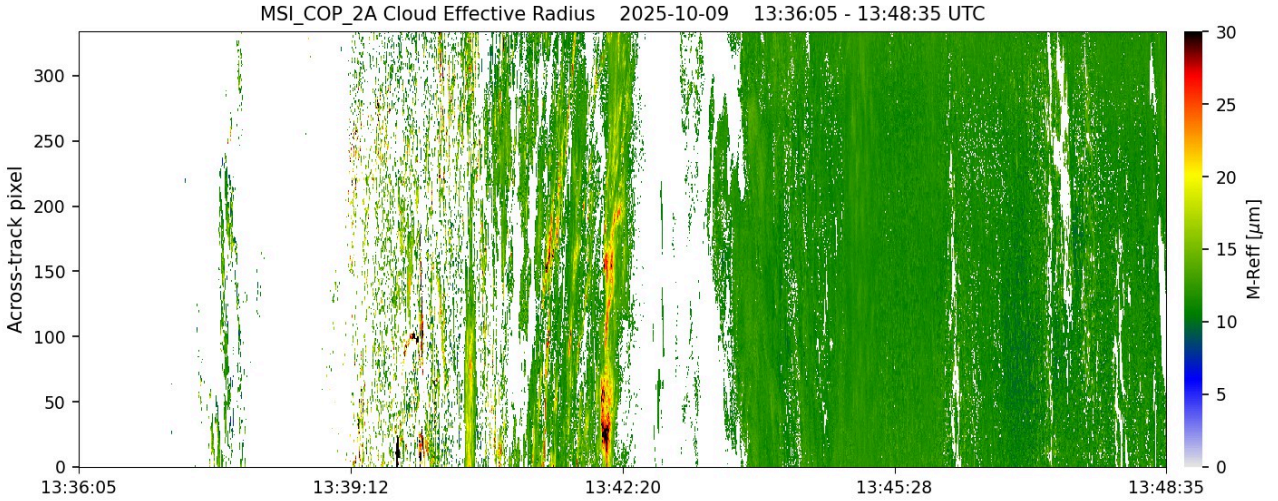
- **Want a quick look?** Use the quicklook or thumbnail to preview the data.
- **Need to analyze?** Work with the enclosure files as demonstrated in the scripts later on.
- **Don't need everything?** Avoid the .zip unless you really need to download all files.
- **Curious about metadata?** Open the XML/JSON metadata files for detailed info.

Quicklook of the data

You don't need to authenticate or authorize to preview the data.
By referencing the `thumbnail` pr `quicklook_jpeg` asset, you're accessing a remote URL where a quicklook image of the product is stored. Note that not all products necessarily have a quicklook stored. This provides a fast and convenient way to visually inspect the data before downloading or processing it.

```
ql_url = ec_product.assets['thumbnail'].href
display(Image(url= ql_url))
```





Token

You can generate an offline access token that is valid for 90 days here. We recommend to save this personal offline token in a separate file and not paste it into your scripts. This offline access token in combination with a client ID and secret creates a refresh token which authenticates/authorizes you to access the data files. In simple terms, the offline token is like a temporary key that helps you securely log in without needing to re-enter your credentials every time.

```
# offline token link: https://portal.maap.eo.esa.int/ini/services/auth/token/9
# CLIENT_ID and CLIENT_SECRET are valid permanently for EVERYONE, but you need
CLIENT_ID="offline-token"
CLIENT_SECRET="p1eL7uonXs6MDxtGbgKdPVRAmnGxHpVE"
# Save your personal offline token in a text file token_yourname.txt in the sa
```

```
# TODO: Create (and rename) token_yourname.txt with your own offline token and
if pathlib.Path("token_yourname.txt").exists():
  with open("token_yourname.txt","rt") as f:
    OFFLINE_TOKEN = f.read().strip().replace("\n","")
    url = "https://iam.maap.eo.esa.int/realms/esa-maap/protocol/openid-connect
    data = {
        "client_id": CLIENT_ID,
        "client_secret": CLIENT_SECRET,
        "grant_type": "refresh_token",
        "refresh_token": OFFLINE_TOKEN,
        "scope": "offline_access openid"
    }

    response = requests.post(url, data=data)
    response.raise_for_status()

    response_json = response.json()
    access_token = response_json.get('access_token')
    print("Access token retrieved successfully.")

    if not access_token:
        raise RuntimeError("Failed to retrieve access token from IAM response"
```

```
Access token retrieved successfully.
```

> ⚠️ The .h5 file is now stored under `enclosure_h5` while the header file is stored in the `enclosure_hdr` asset.

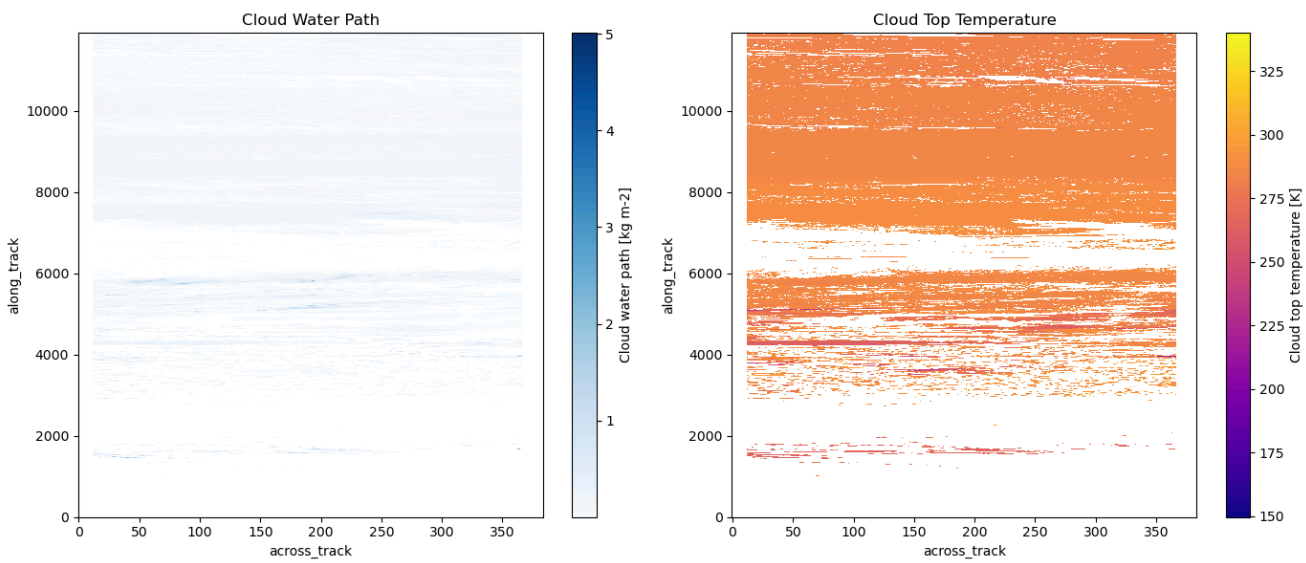
```
# Fetching the url of the desired file
ds_url = ec_product.assets.get('enclosure_h5').href  # Updated to enclosure_h5
print(f"Dataset URL: {ds_url}")
```

```
Dataset URL: https://catalog.maap.eo.esa.int/data/earthcare-pdgs-01/EarthCARE/I
```

This provides a faster way to access the NetCDF data in the cloud. It configures how the HDF5 (.h5) files are read and cached.

```python
io_params = {
    "fsspec_params": {
        "cache_type": "blockcache",
        "block_size": 8 * 1024 * 1024
    },
    "h5py_params": {
        "driver_kwds": {
            "rdcc_nbytes": 8 * 1024 * 1024
        }
    }
}
```

```python
fs = fsspec.filesystem(
    "https",
    headers={"Authorization": f"Bearer {access_token}"},
    **io_params["fsspec_params"]  )

# Open the file and read it into an xarray Dataset
with fs.open(ds_url, "rb") as f:
    ds = xr.open_dataset(f,
                         engine="h5netcdf",
                         **io_params["h5py_params"],
                         group="ScienceData")

    # Do something with ds! Here we plot two variables as an example.
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # Plot Cloud Water Path
    ds["cloud_water_path"].plot(ax=axes[0], cmap="Blues")
    axes[0].set_title("Cloud Water Path")

    # Plot Cloud Top Temperature
    ds["cloud_top_temperature"].plot(ax=axes[1], cmap="plasma")
    axes[1].set_title("Cloud Top Temperature")

    plt.tight_layout()
    plt.show()
```



You can also use your token and the url to download data and not just stream it.

Bulk download : To bulk download data, run the cell below in a loop passing a list of .h5 (or other) urls you fetch from the code previously shown.

```
def download_file_with_bearer_token(url, token, disable_bar=False):
  """
  Downloads a file from a given URL using a Bearer token.
  """

  try:
    headers = {"Authorization": f"Bearer {token}"}
    response = requests.get(url, headers=headers, stream=True)
    response.raise_for_status()  # Raise an exception for bad status codes
    file_size = int(response.headers.get('content-length', 0))

    chunk_size = 8 * 1024 * 1024 # Byes - 1MiB
    file_path = url.rsplit('/', 1)[-1]
    print(file_path)
    with open(file_path, "wb") as f, tqdm(
        desc=file_path,
        total=file_size,
        unit='iB',
        unit_scale=True,
        unit_divisor=1024,
        disable=disable_bar,
      ) as bar:
      for chunk in response.iter_content(chunk_size=chunk_size):
        read_size=f.write(chunk)
        bar.update(read_size)

    if (disable_bar):
      print(f"File downloaded successfully to {file_path}")

  except requests.exceptions.RequestException as e:
    print(f"Error downloading file: {e}")
```

```
import requests
download_file_with_bearer_token(ds_url, access_token)
```

```
ECA_EXBA_MSI_COP_2A_20251009T133627Z_20251009T151530Z_07758E.h5
```

```
ECA_EXBA_MSI_COP_2A_20251009T133627Z_20251009T151530Z_07758E.h5: 100%|
```